// Smart Contract Security Assessment 12.17.2024 - 12.18.2024

VE Governance Updates

Aragon

HALBORN

VE Governance Updates - Aragon

Prepared by: HALBORN

Last Updated 12/30/2024

Date of Engagement by: December 17th, 2024 - December 18th, 2024

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
6	0	0	0	3	3

TABLE OF CONTENTS

1. Introduction

Aragon engaged Halborn to conduct a security assessment on their smart contracts beginning on December 17th, 2024 and ending on December 18th, 2024. The security assessment was scoped to the smart contracts provided to Halborn. Commit hashes and further details can be found in the Scope section of this report.

The Aragon codebase in scope consists of updates made to a set of smart contracts designed to act as a Voting Escrow mechanism.

2. Assessment Summary

Halborn was provided 2 days for the engagement and assigned 1 full-time security engineer to review the security of the smart contracts in scope. The engineer is a blockchain and smart contract security expert with advanced penetration testing and smart contract hacking skills, and deep knowledge of multiple blockchain protocols.

The purpose of the assessment is to:

- Identify potential security issues within the smart contracts.
- Ensure that smart contract functionality operates as intended.

In summary, Halborn identified some improvements to reduce the likelihood and impact of risks, which were mostly addressed by the Aragon team:

- Consider adding a tracking mechanism for the state of the migration and preventing new deposits when migration is active.
- Ensure that the storage layout is compatible with the previous versions to prevent storage collisions.
- Use safeTransferFrom instead of transferFrom in the sweepNFT() function.

3. Test Approach And Methodology

`Halborn` performed a combination of manual review of the code and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of this assessment. While manual testing is recommended to uncover flaws in logic, process, and implementation; automated testing techniques help enhance coverage of smart contracts and can quickly identify items that do not follow security best practices.

The following phases and associated tools were used throughout the term of the assessment:

- Research into architecture, purpose and use of the platform.
- Smart contract manual code review and walkthrough to identify any logic issue.
- Thorough assessment of safety and usage of critical Solidity variables and functions in scope that could led to arithmetic related vulnerabilities.
- Local testing with custom scripts (`Foundry`).
- Fork testing against main networks (`Foundry`).
- Static analysis of security for scoped contract, and imported functions (`Slither`).

4. Caveats

The current security assessment was focused solely on evaluating the changes introduced in the following pull request: https://github.com/aragon/ve-governance/pull/41/files. While the assessment aimed to provide a comprehensive evaluation of the protocol's security posture, it is important to consider the limitations mentioned above when interpreting the findings and recommendations.

5. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

5.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILIY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33

EXPLOITABILIY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

5.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (I:N) Low (I:L) Medium (I:M) High (I:H) Critical (I:C)	0 0.25 0.5 0.75 1

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Integrity (I)	None (I:N) Low (I:L) Medium (I:M) High (I:H) Critical (I:C)	0 0.25 0.5 0.75 1
Availability (A)	None (A:N) Low (A:L) Medium (A:M) High (A:H) Critical (A:C)	0 0.25 0.5 0.75 1
Deposit (D)	None (D:N) Low (D:L) Medium (D:M) High (D:H) Critical (D:C)	0 0.25 0.5 0.75 1
Yield (Y)	None (Y:N) Low (Y:L) Medium (Y:M) High (Y:H) Critical (Y:C)	0 0.25 0.5 0.75 1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

5.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25
Scope (s)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4
Informational	0 - 1.9

6. SCOPE

FILES AND REPOSITORY

(a) Repository: ve-governance

(b) Assessed Commit ID: 0d22c32

(c) Items in scope:

- src/escrow/increasing/Lock.sol
- src/escrow/increasing/VotingEscrowIncreasing.sol
- src/escrow/increasing/interfaces/ILock.sol
- src/escrow/increasing/interfaces/IMigrateable.sol
- src/libs/CurveConstantLib.sol
- src/voting/SimpleGaugeVoter.sol
- aragon/ve-governance/pull/41/files

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:

- https://github.com/aragon/ve-governance/pull/43/files#diff-47d78633ca5fc1c1ad32a426826aa352721d8ffb88aaa8fdf559c3b31cb26f52
- https://github.com/aragon/ve-governance/pull/43/commits/0d58b36801505e2b95c6bae274316dc4d18cf9cf
- https://github.com/aragon/ve-governance/pull/43/commits/5162aa9e35beae934182046ba38decbfb6758233

Out-of-Scope: New features/implementations after the remediation commit IDs.

7. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
0	0	0	3	3

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
DEPOSITS ALLOWED DURING MIGRATION	LOW	SOLVED - 12/27/2024
POSSIBLE STORAGE COLLISIONS IN CONTRACT UPGRADES	LOW	NOT APPLICABLE
POTENTIAL TOKEN LOCKUP AFTER SWEEPING NFT	LOW	RISK ACCEPTED - 12/27/2024
ORDERING FOR NONREENTRANT MODIFIER	INFORMATIONAL	SOLVED - 12/27/2024
TYPOS	INFORMATIONAL	SOLVED - 12/27/2024
FLOATING PRAGMA	INFORMATIONAL	ACKNOWLEDGED - 12/27/2024

8. FINDINGS & TECH DETAILS

8.1 DEPOSITS ALLOWED DURING MIGRATION

// LOW

Description

Currently, the migration implicitly starts by setting the migrator address. After this, the contract still allows deposits from users. This could create an unclear state where users can add deposit during an ending governance period, potentially leading to stranded deposits or participation in a deprecated governance round.

Additionally, although when a withdrawal begins the voting power is set to `0` and the `migrateFor()` function checks for it, there is no explicit prevention or reversal message when attempting migration while a withdrawal is in process.

Code Location

```
119 function migrateFrom(uint256 _tokenId) external returns (uint256 newTo
120     // check the migration contract is set and the tokenid is active
121     if (migrator == address(0)) revert MigrationNotActive();
122     if (!IERC721EMB(lockNFT).isApprovedOrOwner(_msgSender(), _tokenId)
123     if (votingPower(_tokenId) == 0) revert CannotExit();
124 
125     // the user should be approved
126     address owner = IERC721EMB(lockNFT).ownerOf(_tokenId);
127 
128     // reset votes from voting contract
129     if (isVoting(_tokenId)) {
130         ISimpleGaugeVoter(voter).reset(_tokenId);
131     }
132 
133     LockedBalance memory oldLocked = _locked[_tokenId];
134     uint256 value = oldLocked.amount;
135 
136     // burn the current veNFT and write a zero checkpoint.
137     _locked[_tokenId] = LockedBalance(0, 0);
138     totalLocked -= value;
139     _checkpointClear(_tokenId);
140     IERC721EMB(lockNFT).burn(_tokenId);
141 
142     // createLockFor on the new contract for the owner of the veNFT
143     newTokenId = VotingEscrow(migrator).migrateTo(value, owner);
144 
```

```
145         // emit the migrated event
146         emit Migrated(owner, _tokenId, newTokenId, value);
147
148         return newTokenId;
149     }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (2.5)

Recommendation

Consider adding a tracking mechanism for the state of the migration and preventing new deposits when migration is active. Additionally, it is recommended to explicitly revert with a clear message when users attempt to migrate their tokens during the withdrawal process.

Remediation

SOLVED: The **Aragon team** solved this finding in commit `b1a14b0` by following the mentioned recommendation.

Remediation Hash

https://github.com/aragon/ve-governance/pull/43/files#diff-47d78633ca5fc1c1ad32a426826aa352721d8ffb88aaa8fdf559c3b31cb26f52

References

aragon/ve-governance/src/escrow/increasing/VotingEscrowIncreasing.sol#L119-L149

8.2 POSSIBLE STORAGE COLLISIONS IN CONTRACT UPGRADES

// LOW

Description

Throughout the codebase, there are several instances where the storage layout is changed between versions. This could lead to storage collisions when upgrading the contracts, potentially causing data loss or unexpected behavior.

When upgrading to new versions of the contracts, it is important to ensure that the storage layout is compatible with the previous version to prevent storage collisions. This is usually done by adding storage gaps, which are a convention for reserving storage slots in a base contract, allowing future versions of that contract to use up those slots without affecting the storage layout of child contracts.

However, once the `__gap` variable is added and set, it is recommended to not remove variables from the contract, or adding gaps, as this could lead to storage collisions. Instead, variables can be marked as deprecated or unused to prevent them from being used in the contract logic. In the following contracts, the storage gap has changed as follows:

In scope changes:

- `Lock`: The gap went from non-existent to 47.
- `VotingEscrowIncreasing`: The gap went from 42 to 38.

Out of scope changes:

- `Clock`: The gap went from 49 to 50.
- `QuadraticIncreasingEscrow`: The gap went from 44 to 45.
- `ExitQueue`: The gap went from 44 to 46.

For more reference about upgradeability best practices, see here and here.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (2.5)

Recommendation

When upgrading the contracts, ensure that the storage layout is compatible with the previous version to prevent storage collisions. This can be done by following OpenZeppelin's recommended plugins from **their official documentation**:

To help determine the proper storage gap size in the new version of your contract, you can simply attempt an upgrade using upgradeProxy or just run the validations with validateUpgrade (see docs for Hardhat Upgrades or Foundry Upgrades). If a storage gap is not being reduced properly, you will see an error message indicating the expected size of the storage gap.

Alternatively, consider upgrading to a more recent OpenZeppelin version that uses Namespaced Storage Layouts, which handles storage layout changes between versions more efficiently and safely.

Remediation

NOT APPLICABLE: After internal discussion with the **Aragon team**, it was confirmed that the storage layout is consistent with the v1 of the contracts. New changes don't implement any unexpected increases.

References

aragon/ve-governance/src/escrow/increasing/VotingEscrowIncreasing.sol#L484
aragon/ve-governance/src/escrow/increasing/Lock.sol#L171
aragon/ve-governance/src/escrow/increasing/QuadraticIncreasingEscrow.sol#L351
aragon/ve-governance/src/clock/Clock.sol#L225
aragon/ve-governance/src/escrow/increasing/ExitQueue.sol#L229

8.3 POTENTIAL TOKEN LOCKUP AFTER SWEEPING NFT

// LOW

Description

The `sweepNFT()` function from the `VotingEscrowIncreasing` contract uses the `transferFrom()` method for sweeping tokens out of the contract. However, the `transferFrom()` method does not check if the recipient is a contract that can handle the NFT. If the recipient is a contract that does not implement the `onERC721Received()` function, the NFT could be stuck in the contract.

Code Location

```
458 function sweepNFT(uint256 _tokenId, address _to) external nonReentrant
459     // if the token id is not in the contract, revert
460     if (IERC721EMB(lockNFT).ownerOf(_tokenId) != address(this)) revert
461
462     // if the token id is in the queue, we cannot sweep it
463     if (IExitQueue(queue).ticketHolder(_tokenId) != address(0)) revert
464
465     IERC721EMB(lockNFT).transferFrom(address(this), _to, _tokenId);
466     emit SweepNFT(_to, _tokenId);
467 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (2.5)

Recommendation

Use `safeTransferFrom` instead of `transferFrom` in the `sweepNFT()` function of the `VotingEscrow` contract to ensure that the receiver can handle the NFT. In this case, it is worth noting that the receiver can put any logic inside the `onERC721Received()` function, possibly leading to reentrancy behavior, so it is important to make sure all crucial functions of the system are protected against this.

Remediation

RISK ACCEPTED: The **Aragon team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

As the NFT is non-transferrable, by default the DAO needs to also authorize a transfer before the NFT can be sent back to the address via sweep. We assume the DAO will check the address is capable of interacting with the NFT during this authorization.

References

aragon/ve-governance/src/escrow/increasing/VotingEscrowIncreasing.sol#L458-L467

8.4 ORDERING FOR NONREENTRANT MODIFIER

// INFORMATIONAL

Description

In Solidity, if a function has multiple modifiers, they are executed in the order specified. If checks or logic of modifiers depend on other modifiers, this has to be considered in their ordering.

Several functions of the contracts in scope have multiple modifiers, with one of them being `nonReentrant` which prevents reentrancy behavior on the functions. Ideally, the `nonReentrant` modifier should be the first one to prevent even the execution of other modifiers in case of reentrancy behavior.

While there is currently no obvious vulnerability with `nonReentrant` not being the first modifier, placing it first ensures that all other modifiers are executed only if the call is non-reentrant. This is a safer practice and can prevent potential issues in future updates or unforeseen scenarios.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N (0.5)

Recommendation

Switch modifier order to consistently place the nonReentrant modifier as the first one to run so that all other modifiers are executed only if the call is non-reentrant.

Remediation

SOLVED: The **Aragon team** solved this finding in commit `0d58b36` by following the mentioned recommendation.

Remediation Hash

https://github.com/aragon/ve-governance/pull/43/commits/0d58b36801505e2b95c6bae274316dc4d18cf9cf

References

aragon/ve-governance/src/escrow/increasing/Lock.sol#L125-L132

8.5 TYPOS

// INFORMATIONAL

Description

Throughout the codebase, there are several instances of typos in comments. While these typos do not affect the functionality of the code, they can make the codebase harder to read and understand. It is recommended to fix these typos to improve the readability of the codebase.

Instances of this issue include:

- The word `underlying` is misspelled, as `underying` in the `VotingEscrow` contract.
- The word `migration` is misspelled, as `migrationg` in the `VotingEscrow` contract.
- The word `maximum` is misspelled, as `maxiumum` in the `CurveConstantLib` library.

Score

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to fix all typos to improve the readability of the codebase.

Remediation

SOLVED: The **Aragon team** solved this finding in commit `5162aa9` by following the mentioned recommendation.

Remediation Hash

https://github.com/aragon/ve-governance/pull/43/commits/5162aa9e35beae934182046ba38decbfb6758233

References

aragon/ve-governance/src/escrow/increasing/VotingEscrowIncreasing.sol#L71
aragon/ve-governance/src/escrow/increasing/VotingEscrowIncreasing.sol#L118

8.6 FLOATING PRAGMA

// INFORMATIONAL

Description

All contracts in scope currently use floating pragma versions `^0.8.17` which means that the code can be compiled by any compiler version that is greater than or equal to `0.8.17`, and less than `0.9.0`.

However, it is recommended that contracts should be deployed with the same compiler version and flags used during development and testing. Locking the pragma helps to ensure that contracts do not accidentally get deployed using another pragma. For example, an outdated pragma version might introduce bugs that affect the contract system negatively.

Score

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Lock the pragma version to the same version used during development and testing.

Remediation

ACKNOWLEDGED: The **Aragon team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

Fixed pragma can cause difficulties integrating with external codebases. We prefer to use floating.

References

aragon/ve-governance/src/escrow/increasing/Lock.sol#L2
aragon/ve-governance/src/escrow/increasing/VotingEscrowIncreasing.sol#L2
aragon/ve-governance/src/voting/SimpleGaugeVoter.sol#L2

STATIC ANALYSIS REPORT

Description

Halborn used automated testing techniques to enhance the coverage of certain areas of the smart contracts in scope. Among the tools used was Slither, a Solidity static analysis framework. After Halborn verified the smart contracts in the repository and was able to compile them correctly into their abis and binary format, Slither was run against the contracts. This tool can statically verify mathematical relationships between Solidity variables to detect invalid or inconsistent usage of the contracts' APIs across the entire code-base.
The security team assessed all findings identified by the Slither software, however, findings with related to external dependencies are not included in the below results for the sake of report readability.

Output

The findings obtained as a result of the Slither scan were reviewed, and the majority were not included in the report because they were determined as false positives.

```
INFO:Detectors:
Clock.resolveElapsedInEpoch(uint256) (src/clock/Clock.sol#79-83) uses a weak PRNG: "timestamp % EPOCH_DURATION (src/clock/Clock.sol#81)"
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#weak-PRNG
INFO:Detectors:
ExitQueue.withdraw(uint256) (src/escrow/increasing/ExitQueue.sol#122-125) ignores return value by underlying.transfer(msg.sender,_amount) (src/escrow/increasing/ExitQueue.sol#124)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#unchecked-transfer
INFO:Detectors:
CheckpointsUpgradeable._insert(CheckpointsUpgradeable.Checkpoint[],uint32,uint224) (lib/openzeppelin-contracts-upgradeable/contracts/utils/CheckpointsUpgradeable.sol#130-151) uses a dangerous strict equality:
        - last._blockNumber == key (lib/openzeppelin-contracts-upgradeable/contracts/utils/CheckpointsUpgradeable.sol#141)
Clock.resolveEpochStartsIn(uint256) (src/clock/Clock.sol#91-96) uses a dangerous strict equality:
        - (elapsed == 0) (src/clock/Clock.sol#94)
QuadraticIncreasingEscrow._getPastTokenPointInterval(uint256,uint256) (src/escrow/increasing/QuadraticIncreasingEscrow.sol#193-218) uses a dangerous strict equality:
        - tokenPoint.checkpointTs == _timestamp (src/escrow/increasing/QuadraticIncreasingEscrow.sol#209)
VotingEscrow.beginWithdrawal(uint256) (src/escrow/increasing/VotingEscrowIncreasing.sol#368-383) uses a dangerous strict equality:
        - votingPower(_tokenId) == 0 (src/escrow/increasing/VotingEscrowIncreasing.sol#373)
VotingEscrow.migrateFrom(uint256) (src/escrow/increasing/VotingEscrowIncreasing.sol#112-144) uses a dangerous strict equality:
        - votingPower(_tokenId) == 0 (src/escrow/increasing/VotingEscrowIncreasing.sol#116)
VotingEscrow.sweep() (src/escrow/increasing/VotingEscrowIncreasing.sol#417-429) uses a dangerous strict equality:
        - excess == 0 (src/escrow/increasing/VotingEscrowIncreasing.sol#424)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#dangerous-strict-equalities
INFO:Detectors:
Reentrancy in VotingEscrow.withdraw(uint256) (src/escrow/increasing/VotingEscrowIncreasing.sol#386-414):
        External calls:
        - fee = IExitQueue(queue).exit(_tokenId) (src/escrow/increasing/VotingEscrowIncreasing.sol#400)
        - IERC20Upgradeable(token).safeTransfer(address(queue),fee) (src/escrow/increasing/VotingEscrowIncreasing.sol#402)
        State variables written after the call(s):
        - _locked[_tokenId] = LockedBalance(0,0) (src/escrow/increasing/VotingEscrowIncreasing.sol#406)
        VotingEscrow._locked (src/escrow/increasing/VotingEscrowIncreasing.sol#58) can be used in cross function reentrancies:
        - VotingEscrow.locked(uint256) (src/escrow/increasing/VotingEscrowIncreasing.sol#269-271)
        - VotingEscrow.migrateFrom(uint256) (src/escrow/increasing/VotingEscrowIncreasing.sol#112-144)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-1
INFO:Detectors:
QuadraticIncreasingEscrow._checkpoint(uint256,ILockedBalanceIncreasing.LockedBalance,ILockedBalanceIncreasing.LockedBalance).uNew (src/escrow/increasing/QuadraticIncreasingEscrow.sol#258) is a local variable never initialized
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#uninitialized-local-variables
INFO:Detectors:
VotingEscrow.enableMigration(address) (src/escrow/increasing/VotingEscrowIncreasing.sol#100-106) ignores return value by IERC20Upgradeable(token).approve(migrator,type()(uint256).max) (src/escrow/increasing/VotingEscrowIncreasing.sol#104)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#unused-return
INFO:Detectors:
Lock.initialize(address,string,string,address)._name (src/escrow/increasing/Lock.sol#54) shadows:
        - ERC721Upgradeable._name (lib/openzeppelin-contracts-upgradeable/contracts/token/ERC721/ERC721Upgradeable.sol#25) (state variable)
Lock.initialize(address,string,string,address)._symbol (src/escrow/increasing/Lock.sol#54) shadows:
        - ERC721Upgradeable._symbol (lib/openzeppelin-contracts-upgradeable/contracts/token/ERC721/ERC721Upgradeable.sol#28) (state variable)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#local-variable-shadowing
INFO:Detectors:
ExitQueue.initialize(address,uint48,address,uint256,address,uint48)._escrow (src/escrow/increasing/ExitQueue.sol#58) lacks a zero-check on :
                - escrow = _escrow (src/escrow/increasing/ExitQueue.sol#60)
ExitQueue.initialize(address,uint48,address,uint256,address,uint48)._clock (src/escrow/increasing/ExitQueue.sol#58) lacks a zero-check on :
                - clock = _clock (src/escrow/increasing/ExitQueue.sol#61)
Lock.initialize(address,string,string,address)._escrow (src/escrow/increasing/Lock.sol#54) lacks a zero-check on :
                - escrow = _escrow (src/escrow/increasing/Lock.sol#58)
QuadraticIncreasingEscrow.initialize(address,address,uint48,address)._escrow (src/escrow/increasing/QuadraticIncreasingEscrow.sol#73) lacks a zero-check on :
                - escrow = _escrow (src/escrow/increasing/QuadraticIncreasingEscrow.sol#74)
QuadraticIncreasingEscrow.initialize(address,address,uint48,address)._clock (src/escrow/increasing/QuadraticIncreasingEscrow.sol#73) lacks a zero-check on :
                - clock = _clock (src/escrow/increasing/QuadraticIncreasingEscrow.sol#76)
VotingEscrow.enableMigration(address)._migrator (src/escrow/increasing/VotingEscrowIncreasing.sol#100) lacks a zero-check on :
                - migrator = _migrator (src/escrow/increasing/VotingEscrowIncreasing.sol#102)
VotingEscrow.initialize(address,address,address,uint256)._clock (src/escrow/increasing/VotingEscrowIncreasing.sol#163) lacks a zero-check on :
                - clock = _clock (src/escrow/increasing/VotingEscrowIncreasing.sol#170)
VotingEscrow.setCurve(address)._curve (src/escrow/increasing/VotingEscrowIncreasing.sol#180) lacks a zero-check on :
                - curve = _curve (src/escrow/increasing/VotingEscrowIncreasing.sol#181)
VotingEscrow.setVoter(address)._voter (src/escrow/increasing/VotingEscrowIncreasing.sol#185) lacks a zero-check on :
```

```
                - voter = _voter (src/escrow/increasing/VotingEscrowIncreasing.sol#186)
VotingEscrow.setQueue(address)._queue (src/escrow/increasing/VotingEscrowIncreasing.sol#190) lacks a zero-check on :
                - queue = _queue (src/escrow/increasing/VotingEscrowIncreasing.sol#191)
VotingEscrow.setClock(address)._clock (src/escrow/increasing/VotingEscrowIncreasing.sol#195) lacks a zero-check on :
                - clock = _clock (src/escrow/increasing/VotingEscrowIncreasing.sol#196)
VotingEscrow.setLockNFT(address)._nft (src/escrow/increasing/VotingEscrowIncreasing.sol#202) lacks a zero-check on :
                - lockNFT = _nft (src/escrow/increasing/VotingEscrowIncreasing.sol#204)
SimpleGaugeVoter.initialize(address,address,bool,address)._escrow (src/voting/SimpleGaugeVoter.sol#46) lacks a zero-check on :
                - escrow = _escrow (src/voting/SimpleGaugeVoter.sol#50)
SimpleGaugeVoter.initialize(address,address,bool,address)._clock (src/voting/SimpleGaugeVoter.sol#46) lacks a zero-check on :
                - clock = _clock (src/voting/SimpleGaugeVoter.sol#51)
SimpleGaugeVoterSetup.constructor(address,address,address,address,address,address)._voterBase (src/voting/SimpleGaugeVoterSetup.sol#79) lacks a zero-check on :
                - voterBase = _voterBase (src/voting/SimpleGaugeVoterSetup.sol#80)
SimpleGaugeVoterSetup.constructor(address,address,address,address,address,address)._curveBase (src/voting/SimpleGaugeVoterSetup.sol#79) lacks a zero-check on :
                - curveBase = _curveBase (src/voting/SimpleGaugeVoterSetup.sol#81)
SimpleGaugeVoterSetup.constructor(address,address,address,address,address,address)._queueBase (src/voting/SimpleGaugeVoterSetup.sol#79) lacks a zero-check on :
                - queueBase = _queueBase (src/voting/SimpleGaugeVoterSetup.sol#82)
SimpleGaugeVoterSetup.constructor(address,address,address,address,address,address)._escrowBase (src/voting/SimpleGaugeVoterSetup.sol#79) lacks a zero-check on :
                - escrowBase = _escrowBase (src/voting/SimpleGaugeVoterSetup.sol#83)
SimpleGaugeVoterSetup.constructor(address,address,address,address,address,address)._clockBase (src/voting/SimpleGaugeVoterSetup.sol#79) lacks a zero-check on :
                - clockBase = _clockBase (src/voting/SimpleGaugeVoterSetup.sol#84)
SimpleGaugeVoterSetup.constructor(address,address,address,address,address,address)._nftBase (src/voting/SimpleGaugeVoterSetup.sol#79) lacks a zero-check on :
                - nftBase = _nftBase (src/voting/SimpleGaugeVoterSetup.sol#85)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-address-validation
INFO:Detectors:
Reentrancy in VotingEscrow.migrateFrom(uint256) (src/escrow/increasing/VotingEscrowIncreasing.sol#112-144):
        External calls:
        - ISimpleGaugeVoter(voter).reset(_tokenId) (src/escrow/increasing/VotingEscrowIncreasing.sol#127)
        State variables written after the call(s):
        - _locked[_tokenId] = LockedBalance(0,0) (src/escrow/increasing/VotingEscrowIncreasing.sol#134)
        - totalLocked -= value (src/escrow/increasing/VotingEscrowIncreasing.sol#135)
Reentrancy in VotingEscrow.withdraw(uint256) (src/escrow/increasing/VotingEscrowIncreasing.sol#386-414):
        External calls:
        - fee = IExitQueue(queue).exit(_tokenId) (src/escrow/increasing/VotingEscrowIncreasing.sol#400)
        - IERC20Upgradeable(token).safeTransfer(address(queue),fee) (src/escrow/increasing/VotingEscrowIncreasing.sol#402)
        State variables written after the call(s):
        - totalLocked -= value (src/escrow/increasing/VotingEscrowIncreasing.sol#407)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-2
INFO:Detectors:
Reentrancy in VotingEscrow.enableMigration(address) (src/escrow/increasing/VotingEscrowIncreasing.sol#100-106):
        External calls:
        - IERC20Upgradeable(token).approve(migrator,type()(uint256).max) (src/escrow/increasing/VotingEscrowIncreasing.sol#104)
        Event emitted after the call(s):
        - MigrationEnabled(_migrator) (src/escrow/increasing/VotingEscrowIncreasing.sol#105)
Reentrancy in VotingEscrow.migrateFrom(uint256) (src/escrow/increasing/VotingEscrowIncreasing.sol#112-144):
        External calls:
        - ISimpleGaugeVoter(voter).reset(_tokenId) (src/escrow/increasing/VotingEscrowIncreasing.sol#127)
        - _checkpointClear(_tokenId) (src/escrow/increasing/VotingEscrowIncreasing.sol#136)
                - IEscrowCurveIncreasing(curve).checkpoint(_tokenId,LockedBalance(0,0),LockedBalance(0,checkpointClearTime.toUint48())) (src/escrow/increasing/VotingEscrowIncreasing.sol#351)
        - IERC721EnumerableMintableBurnable(lockNFT).burn(_tokenId) (src/escrow/increasing/VotingEscrowIncreasing.sol#137)
        - newTokenId = VotingEscrow(migrator).migrateTo(value,owner) (src/escrow/increasing/VotingEscrowIncreasing.sol#140)
        Event emitted after the call(s):
        - Migrated(owner,_tokenId,newTokenId,value) (src/escrow/increasing/VotingEscrowIncreasing.sol#142)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-3
INFO:Detectors:
Void constructor called in SimpleGaugeVoterSetup.constructor(address,address,address,address,address,address) (src/voting/SimpleGaugeVoterSetup.sol#79-86):
        - PluginSetup() (src/voting/SimpleGaugeVoterSetup.sol#79)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#void-constructor
INFO:Detectors:
SimpleGaugeVoter._reset(uint256) (src/voting/SimpleGaugeVoter.sol#173-209) has costly operations inside a loop:
        - totalVotingPowerCast -= votingPowerToRemove (src/voting/SimpleGaugeVoter.sol#208)
SimpleGaugeVoter._vote(uint256,IGaugeVote.GaugeVote[]) (src/voting/SimpleGaugeVoter.sol#88-165) has costly operations inside a loop:
        - totalVotingPowerCast += votingPowerUsed (src/voting/SimpleGaugeVoter.sol#160)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#costly-operations-inside-a-loop
INFO:Detectors:
Clock (src/clock/Clock.sol#13-226) should inherit from IBeacon (lib/openzeppelin-contracts/contracts/proxy/beacon/IBeacon.sol#9-16)
ExitQueue (src/escrow/increasing/ExitQueue.sol#18-223) should inherit from IBeacon (lib/openzeppelin-contracts/contracts/proxy/beacon/IBeacon.sol#9-16)
Lock (src/escrow/increasing/Lock.sol#14-150) should inherit from IBeacon (lib/openzeppelin-contracts/contracts/proxy/beacon/IBeacon.sol#9-16)
Lock (src/escrow/increasing/Lock.sol#14-150) should inherit from IERC721EnumerableMintableBurnable (src/escrow/increasing/interfaces/IERC721EMB.sol#6-12)
QuadraticIncreasingEscrow (src/escrow/increasing/QuadraticIncreasingEscrow.sol#23-314) should inherit from IBeacon (lib/openzeppelin-contracts/contracts/proxy/beacon/IBeacon.sol#9-16)
VotingEscrow (src/escrow/increasing/VotingEscrowIncreasing.sol#28-464) should inherit from IBeacon (lib/openzeppelin-contracts/contracts/proxy/beacon/IBeacon.sol#9-16)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-inheritance
INFO:Detectors:
The following unused import(s) in src/factory/GaugesDaoFactory.sol should be removed:
        -import {IWithdrawalQueueErrors} from "src/escrow/increasing/interfaces/IVotingEscrowIncreasing.sol"; (src/factory/GaugesDaoFactory.sol#7)

        -import {IGaugeVote} from "src/voting/ISimpleGaugeVoter.sol"; (src/factory/GaugesDaoFactory.sol#8)

        -import {IEscrowCurveTokenStorage} from "@escrow-interfaces/IEscrowCurveIncreasing.sol"; (src/factory/GaugesDaoFactory.sol#6)
The following unused import(s) in src/voting/SimpleGaugeVoterSetup.sol should be removed:
        -import {IDAO} from "@aragon/osx/core/dao/IDAO.sol"; (src/voting/SimpleGaugeVoterSetup.sol#8)

        -import {IProposal} from "@aragon/osx/core/plugin/proposal/IProposal.sol"; (src/voting/SimpleGaugeVoterSetup.sol#10)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#unused-imports
INFO:Detectors:
PluginRepoFactory.pluginRepoBase (lib/osx/packages/contracts/src/framework/plugin/repo/PluginRepoFactory.sol#22) should be immutable
PluginRepoFactory.pluginRepoRegistry (lib/osx/packages/contracts/src/framework/plugin/repo/PluginRepoFactory.sol#19) should be immutable
PluginSetupProcessor.repoRegistry (lib/osx/packages/contracts/src/framework/plugin/setup/PluginSetupProcessor.sol#128) should be immutable
SimpleGaugeVoterSetup.clockBase (src/voting/SimpleGaugeVoterSetup.sol#73) should be immutable
SimpleGaugeVoterSetup.curveBase (src/voting/SimpleGaugeVoterSetup.sol#64) should be immutable
SimpleGaugeVoterSetup.escrowBase (src/voting/SimpleGaugeVoterSetup.sol#70) should be immutable
SimpleGaugeVoterSetup.nftBase (src/voting/SimpleGaugeVoterSetup.sol#76) should be immutable
SimpleGaugeVoterSetup.queueBase (src/voting/SimpleGaugeVoterSetup.sol#67) should be immutable
SimpleGaugeVoterSetup.voterBase (src/voting/SimpleGaugeVoterSetup.sol#61) should be immutable
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-that-could-be-declared-immutable
```

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.